

S.A.G.E. - Sistema di Analisi e Gestione Economica

Corso: Basi di Dati
Anno accademico 2025/2026

Componenti del gruppo

Antonio Scharmuller
Riccardo Dallacasa

Matricole

Matricola 000000
Matricola 000000

Email universitarie

antonio.scharmuller@studio.unibo.it
riccardo.dallacasa@studio.unibo.it

Docente: Nome Cognome

18 maggio 2026

Indice

1 Introduzione

1.1 Obiettivo del progetto

S.A.G.E. (Sistema di Analisi e Gestione Economica) è un'applicazione desktop per la gestione delle finanze personali di uno studente universitario. Il sistema supporta la registrazione di entrate e spese, la classificazione dei movimenti, la definizione di budget e la consultazione di riepiloghi utili per monitorare l'andamento economico nel tempo.

L'obiettivo del progetto non è soltanto realizzare un'applicazione funzionante, ma documentare l'intero ciclo di progettazione di una base di dati: analisi dei requisiti, modellazione concettuale, traduzione logica, implementazione SQL e collegamento a un programma Java tramite JDBC.

1.2 Descrizione generale del sistema

S.A.G.E. è progettato come sistema multiutente. Ogni studente accede con le proprie credenziali e visualizza soltanto i dati collegati alla propria email. Il portafoglio personale comprende transazioni, categorie, fonti, tag, budget, documenti digitali e modelli di spese ricorrenti.

L'applicazione è realizzata in Java Swing e comunica con un database MySQL tramite JDBC. La logica applicativa è organizzata in livelli: le classi del package `model` rappresentano il dominio, i DAO incapsulano le query SQL, i controller coordinano i casi d'uso e le classi del package `view` compongono l'interfaccia grafica.

1.3 Ambito del sistema

L'ambito del progetto comprende:

- gestione individuale del portafoglio economico personale;
- registrazione di spese ed entrate;
- classificazione delle spese tramite categorie e tag;
- classificazione delle entrate tramite fonti;
- definizione di budget mensili e budget specifici per categoria;
- gestione di spese ricorrenti come modelli astratti;
- associazione di documenti digitali alle spese;
- visualizzazione di riepiloghi e statistiche.

Non rientrano nell'ambito del sistema la condivisione di conti tra utenti, la sincronizzazione cloud dei file allegati e l'integrazione con servizi bancari esterni.

1.4 Utenti e ruoli previsti

Il sistema prevede due ruoli:

- **Utente:** registra transazioni, consulta il proprio portafoglio, gestisce categorie personali, definisce budget e associa documenti alle spese.
- **Amministratore:** gestisce elementi di sistema, come categorie, fonti e tag globali. L'amministratore può accedere solo a statistiche aggregate e non ai dettagli finanziari personali degli utenti.

1.5 Tecnologie utilizzate

- **DBMS:** MySQL.
- **Linguaggio applicativo:** Java.
- **Interfaccia:** Java Swing.
- **Connessione al database:** JDBC con MySQL Connector/J.
- **Architettura:** applicazione desktop 2-tier con accesso diretto al DBMS.

1.6 Scelte progettuali preliminari

Le scelte iniziali più rilevanti sono:

- isolamento dei dati tramite email dell'utente autenticato;
- distinzione tra elementi globali di sistema ed elementi personali;
- gestione opzionale del budget, che non blocca l'inserimento di nuove spese;
- memorizzazione dei documenti digitali tramite percorso file, evitando di salvare il contenuto binario direttamente nel database.

La scelta sui documenti riduce il peso del DBMS e semplifica le operazioni JDBC. Presenta però un limite noto: se l'applicazione viene usata da un altro dispositivo collegato allo stesso database, un percorso locale potrebbe non essere più valido. Questa criticità viene considerata tra i possibili sviluppi futuri.

2 Analisi dei requisiti

2.1 Requisiti in linguaggio naturale

Il sistema deve permettere a uno studente di registrare e monitorare i propri movimenti economici. Ogni transazione deve avere data, importo e descrizione. Una spesa deve essere associata a una categoria e può avere tag e documenti digitali; un'entrata deve essere associata a una fonte. L'utente deve poter definire budget per periodo e, opzionalmente, per categoria. Il sistema deve inoltre supportare spese ricorrenti come modelli da cui generare movimenti effettivi.

2.2 Analisi delle ambiguità e decisioni progettuali

Concetto	Ambiguità	Decisione progettuale
Periodo temporale	Come collegare transazioni e budget mensili?	Il periodo è derivato dalla data della transazione e storicizzato nella tabella PERIODO.
Transazione	Separare fisicamente spese ed entrate?	Si usa una relazione unica TRANSIZIONE con attributo TipoTransazione.
Classificazione	Come distinguere catalogo comune e personalizzazioni?	CATEGORIA, TAG e FONTE usano <code>is_system</code> ed eventuale <code>Email_Proprietario</code> .
Spesa ricorrente	È una transazione già avvenuta?	No: è un modello generatore che può produrre transazioni future.
Documenti	Salvare file o riferimento?	Il DB salva percorso, tipo e data acquisizione. Il file resta nel file system, non in un BLOB.
Privacy	L'admin vede i movimenti personali?	No: l'accesso amministrativo è limitato a dati aggregati.

2.3 Specifiche ristrutturate

- R1: autenticare utenti e amministratori.
- R2: registrare spese ed entrate con data, importo e descrizione.
- R3: associare ogni spesa a una categoria e ogni entrata a una fonte.
- R4: associare tag e documenti digitali alle spese.
- R5: gestire budget mensili e budget per categoria.
- R6: gestire spese ricorrenti come modelli astratti.

- R7: filtrare ogni dato operativo tramite l'email dell'utente autenticato.
- R8: consentire all'amministratore solo operazioni su cataloghi globali e statistiche aggregate.

2.4 Glossario dei termini essenziali

- **Utente:** studente identificato da email, nome, cognome, password e ruolo.
- **Transazione:** movimento economico generico, distinto in spesa o entrata.
- **Categoria:** classificazione principale di una spesa.
- **Fonte:** origine di un'entrata.
- **Tag:** etichetta trasversale che consente classificazioni aggiuntive.
- **Budget:** limite di spesa collegato a utente, periodo ed eventuale categoria.
- **Documento:** riferimento digitale a uno scontrino o a una fattura associata a una spesa.
- **Spesa ricorrente:** modello da cui generare spese periodiche.

2.5 Classi di utenza e viste del sistema

- **Studente:** utilizza dashboard, transazioni, budget, documenti, categorie personali e report.
- **Amministratore:** gestisce cataloghi globali e consulta indicatori aggregati.

La vista studente è centrata sul portafoglio personale. Ogni schermata applicativa riceve l'utente corrente e usa la sua email come chiave di sessione.

2.6 Elenco delle operazioni principali

- Login e apertura della dashboard personale.
- Inserimento di una spesa con categoria, tag opzionali e documento opzionale.
- Inserimento di una entrata con fonte.
- Consultazione dei movimenti per periodo.
- Definizione o aggiornamento di un budget.
- Associazione e apertura di un documento digitale.
- Creazione di categorie, fonti e tag personali.
- Consultazione di riepiloghi e statistiche aggregate.

2.7 Vincoli espliciti e impliciti

- Ogni query sui dati personali deve filtrare per email dell'utente autenticato.
- Una spesa deve avere una categoria e non deve avere una fonte.
- Una entrata deve avere una fonte e non deve avere una categoria.
- I tag sono associabili solo alle spese.
- I documenti sono associabili solo alle spese.
- Il budget è opzionale e non impedisce l'inserimento di movimenti.
- Il contenuto dei documenti non è salvato nel database; viene salvato solo il riferimento al file.

3 Progettazione concettuale

3.1 Criteri generali di modellazione

La progettazione concettuale privilegia un modello chiaro, con entità separate per gli oggetti persistenti principali e associazioni esplicite per i legami molti-a-molti. Le scelte sono guidate da tre criteri: isolamento dei dati per utente, semplicità di interrogazione e coerenza con l'applicazione Java.

3.2 Entità principali

Entità	Ruolo nel sistema
UTENTE	Identifica studenti e amministratori. L'email è chiave primaria e riferimento per i dati personali.
TRANSIZIONE	Rappresenta movimenti economici, sia spese sia entrate, tramite <code>TipoTransazione</code> .
CATEGORIA	Classifica le spese; può essere di sistema o personale.
FONTE	Classifica le entrate; può essere di sistema o personale.
TAG	Aggiunge classificazioni trasversali alle spese.
PERIODO	Storicizza mese e anno usati dai budget.
BUDGET	Definisce una soglia di spesa per periodo, con eventuale categoria.
DOCUMENTO	Conserva percorso, tipo e data di acquisizione del file associato a una spesa.
SPESA_RICORRENTE	Modello astratto per spese periodiche future.

3.3 Raffinamenti dello schema

3.3.1 Gerarchia delle transazioni

Nel dominio esistono due sottotipi di transazione: spesa ed entrata. La specializzazione è totale ed esclusiva: ogni transazione è necessariamente una spesa o una entrata, e non può essere entrambe. Nel modello logico questa gerarchia è collassata in `TRANSIZIONE`, con l'attributo `TipoTransazione`. I vincoli `CHECK` garantiscono la coerenza: una spesa richiede `ID_Categoria`, una entrata richiede `ID_Fonte`.

3.3.2 Elementi di sistema e personali

`CATEGORIA`, `TAG` e `FONTE` sono modellati con una soluzione ibrida. Gli elementi globali hanno `is_system = TRUE` e nessun proprietario; gli elementi personali hanno `is_system = FALSE` e sono collegati a un utente. Le query applicative mostrano a uno studente sia gli elementi globali sia quelli personali.

3.3.3 Reificazione del budget

Il budget è modellato come entità autonoma collegata a utente e periodo. L'eventuale categoria distingue un budget specifico da un budget mensile complessivo. L'attributo `Totale_Speso_Attuale` è ridondante, ma serve a rendere efficiente la dashboard.

3.3.4 Documenti digitali

Il documento digitale è collegato a una spesa e contiene `Path_File`, `Tipo_File` e `Data_Acquisizione_Documento`. Il contenuto binario del file non è salvato nel DBMS.

Questa scelta riduce il peso del database e semplifica backup, query e codice JDBC. Il limite principale è la portabilità: spostandosi su un altro dispositivo, il percorso locale potrebbe non esistere. Una futura versione potrebbe usare path relativi sincronizzati oppure un attributo `LONGBLOB`, qualora la portabilità completa del documento attraverso il solo database diventasse un requisito prioritario.

3.4 Associazioni principali e cardinalità

Associazione	Cardinalità	Motivazione
Utente–Transazione	1:N	Un utente può registrare molte transazioni; ogni transazione appartiene a un solo utente.
Transazione–Categoria	N:1 opzionale	Solo le spese hanno una categoria.
Transazione–Fonte	N:1 opzionale	Solo le entrate hanno una fonte.
Spesa–Tag	N:M	Una spesa può avere più tag e un tag può essere riutilizzato.
Spesa–Documento	1:N logico	Una spesa può avere documenti; nella UI corrente si limita a un documento per semplicità.
Utente–Budget	1:N	Ogni utente può definire più budget nel tempo.
Periodo–Budget	1:N	Un periodo può essere condiviso da budget di utenti diversi.

3.5 Vincoli non rappresentabili direttamente nello schema E/R

Alcune regole richiedono vincoli SQL o controlli applicativi:

- una categoria personale è visibile solo al proprietario;
- una spesa può usare solo categorie di sistema o categorie personali dell'utente;
- un documento può essere collegato solo a una spesa, non a una entrata;
- l'amministratore non deve visualizzare dettagli finanziari personali;
- l'inserimento di spesa, tag, documento e aggiornamento budget deve essere atomico.

3.6 Dizionario di entità e associazioni

Il dizionario riassume gli attributi principali:

- UTENTE: Email, Password, Nome, Cognome, Ruolo.
- TRANSIZIONE: ID, TipoTransazione, Importo, Data, Descrizione, Email, Categoria/Fonte.
- BUDGET: ID, Importo_Limite, Alert_Soglia, Totale_Speso_Attuale, Periodo, Categoria, Email.
- DOCUMENTO: ID, ID_Transizione, Path_File, Tipo_File, Data_Acquisizione_Documento.
- SPESA_TAG: associazione multi-a-multi tra spesa e tag.

4 Specifiche funzionali

4.1 Elenco delle funzionalità richieste

Le funzionalità principali sono organizzate in operazioni utente e operazioni amministrative. Le prime lavorano sempre sul portafoglio dello studente autenticato; le seconde riguardano cataloghi e statistiche aggregate.

4.2 Operazioni di inserimento

Operazione	Input	Tabelle coinvolte
Login	Email, password	UTENTE
Nuova spesa	Importo, data, descrizione, categoria, tag, documento opzionale	PERIODO, TRANSIZIONE, SPESA_TAG, DOCUMENTO, BUDGET

Operazione	Input	Tabelle coinvolte
Nuova entrata	Importo, data, descrizione, fonte	PERIODO, TRANSIZIONE
Nuovo budget	Periodo, importo limite, categoria opzionale	BUDGET, PERIODO, TRANSIZIONE
Nuovo documento	Spesa, percorso file, tipo file	DOCUMENTO, TRANSIZIONE

4.3 Operazioni di modifica ed eliminazione

Le modifiche riguardano principalmente budget, categorie personali, fonti personali e tag personali. Le eliminazioni devono rispettare i vincoli referenziali: ad esempio una categoria personale eliminata non deve lasciare riferimenti incoerenti nelle spese già registrate. La cancellazione di transazioni o documenti sfrutta le regole di integrità definite nello schema SQL.

4.4 Operazioni di interrogazione

- elenco delle transazioni dell'utente per periodo;
- elenco di categorie, fonti e tag disponibili;
- elenco dei budget e relativo stato di superamento;
- elenco dei documenti associati alle spese dell'utente;
- elenco delle spese ancora prive di documento.

4.5 Operazioni statistiche e amministrative

Le statistiche includono totali mensili di spese ed entrate, rapporto percentuale tra spese ed entrate, budget superati e aggregazioni per categoria. L'amministratore deve utilizzare viste aggregate, evitando l'accesso a descrizioni, documenti o movimenti granulari degli utenti.

5 Progettazione logica

5.1 Stima del volume dei dati

Per un utilizzo universitario individuale si stimano volumi contenuti ma crescenti nel tempo:

- poche decine di utenti demo nel contesto del progetto;
- circa 50–150 transazioni annue per utente;
- 5–20 categorie personali per utente;
- un numero di documenti inferiore o uguale al numero di spese;
- pochi budget per mese e per utente.

Questi volumi giustificano un modello relazionale normalizzato e indici mirati sulle chiavi esterne e sui campi più filtrati.

5.2 Descrizione delle operazioni principali e frequenza

Operazione	Frequenza stimata	Osservazioni
Login	Alta	Prima operazione di ogni sessione.
Inserimento transazione	Media	Deve essere atomico con periodo, documento e budget.
Consultazione dashboard	Alta	Motiva la ridondanza sul totale speso del budget.

Operazione	Frequenza sti- mata	Osservazioni
Gestione documenti	Media	Lettura frequente, inserimento occasionale.
Report aggregati	Bassa/Media	Usati per analisi periodica.

5.3 Schemi di navigazione

La navigazione principale parte da **UTENTE**, identificato dalla sessione. Da esso si raggiungono le transazioni, i budget, gli elementi personali e le spese ricorrenti. Le transazioni permettono di navigare verso categorie, fonti, tag e documenti. Questo schema conferma la centralità dell'email come chiave di isolamento.

5.4 Tabelle degli accessi

Funzione	Tabelle lette	Tabelle scritte
Login	UTENTE	–
Nuova spesa	PERIODO, BUDGET	PERIODO, TRANSIZIONE, SPESA_TAG, DOCUMENTO, BUDGET
Nuova entrata	PERIODO	PERIODO, TRANSIZIONE
Documenti	DOCUMENTO, TRANSIZIONE	DOCUMENTO
Dashboard budget	BUDGET, PERIODO, CATEGORIA	–

5.5 Analisi delle ridondanze

L'attributo **Totale_Speso_Attuale** della relazione **BUDGET** è ridondante perché può essere ricavato sommando le spese presenti in **TRANSIZIONE**. La ridondanza viene mantenuta per ridurre il costo delle letture più frequenti della dashboard.

La coerenza viene garantita con due regole applicative:

- quando viene inserita una spesa, l'inserimento della transazione, l'associazione di tag/documento e l'aggiornamento degli eventuali budget avvengono nella stessa transazione JDBC;
- quando un budget viene creato o aggiornato dopo spese già registrate, il valore iniziale viene calcolato con una query di aggregazione sulle transazioni esistenti.

In questo modo il budget rimane opzionale senza perdere consistenza: l'assenza di un budget non blocca la registrazione della spesa, mentre l'esistenza di un budget impone la sincronizzazione del suo totale ridondante.

5.6 Raffinamento dello schema

5.6.1 Eliminazione delle gerarchie

La gerarchia spesa/entrata viene eliminata collassando i sottotipi in **TRANSIZIONE**. La distinzione rimane espressa da **TipoTransazione**, mentre i vincoli **CHECK** controllano la presenza corretta di categoria o fonte.

5.6.2 Eliminazione degli attributi composti

Non sono presenti attributi composti significativi da scomporre ulteriormente. Nome e cognome dell'utente sono già separati; il periodo è rappresentato da mese e anno nella tabella **PERIODO**.

5.6.3 Gestione degli attributi multivalore

I tag associati a una spesa rappresentano un attributo multivalore. Vengono tradotti nella relazione SPESA_TAG, con chiave primaria composta da ID_Transizione e ID_Tag.

5.6.4 Scelta delle chiavi primarie

L'email è scelta come chiave primaria di UTENTE, perché identifica direttamente la sessione applicativa. Le altre entità principali utilizzano identificatori numerici auto-incrementali per semplificare riferimenti e join.

5.6.5 Traduzione delle associazioni

Le associazioni 1:N sono tradotte tramite chiavi esterne. Le associazioni N:M, come quella tra spese e tag, sono tradotte tramite tabella ponte. I vincoli applicativi più specifici sono rafforzati tramite trigger e controlli SQL.

5.7 Schema relazionale finale

Lo schema relazionale finale comprende le relazioni UTENTE, PERIODO, CATEGORIA, FONTE, TAG, TRANSIZIONE, SPESA_TAG, DOCUMENTO, BUDGET e SPESA_RICORRENTE. La struttura è coerente con lo script `schema.sql`, che definisce chiavi primarie, chiavi esterne, vincoli CHECK e indici.

5.8 Vincoli di integrità

I vincoli principali sono:

- integrità referenziale tra transazioni e utenti;
- vincoli CHECK su ruolo, mese, anno, importo e tipo transazione;
- vincolo UNIQUE su PERIODO per la coppia mese/anno;
- vincolo UNIQUE su BUDGET per evitare duplicati sullo stesso ambito;
- vincoli di proprietà sugli elementi personali.

5.9 Controllo della normalizzazione

Le relazioni principali rispettano la terza forma normale: gli attributi dipendono dalla chiave, da tutta la chiave e non da attributi non chiave. L'unica ridondanza intenzionale è Totale_Speso_Attuale in BUDGET, documentata e controllata tramite logica applicativa.

6 Implementazione SQL

6.1 Script di creazione del database

Lo script principale crea il database MySQL, definisce le tabelle, imposta chiavi primarie e chiavi esterne e aggiunge vincoli CHECK. Gli script separati gestiscono popolamento iniziale, trigger, viste e query operative.

6.2 Creazione delle tabelle

Le tabelle sono create partendo dalle entità indipendenti, come UTENTE e PERIODO, per poi introdurre le tabelle dipendenti. Questa sequenza evita problemi con chiavi esterne non ancora disponibili.

6.3 Definizione di chiavi primarie e chiavi esterne

Ogni tabella ha una chiave primaria esplicita. Le chiavi esterne usano azioni coerenti con il dominio: ad esempio la cancellazione di un utente elimina i suoi dati personali, mentre l'eliminazione di un periodo già usato è limitata per preservare la storia dei budget.

6.4 Vincoli NOT NULL, UNIQUE, CHECK

I vincoli NOT NULL garantiscono la presenza degli attributi essenziali. I vincoli UNIQUE evitano duplicati su periodo e budget. I vincoli CHECK formalizzano regole di dominio come tipo transazione, mese valido, importo positivo e coerenza tra categoria/fonte.

6.5 Trigger, procedure o viste

I trigger presenti negli script rafforzano vincoli che non sono esprimibili interamente con semplici CHECK, come la validità di categorie, fonti, tag e documenti rispetto all'utente proprietario e al tipo della transazione. Le viste semplificano interrogazioni frequenti, come il riepilogo dei movimenti e lo stato dei budget.

6.6 Popolamento iniziale del database

6.7 Query SQL delle operazioni principali

6.7.1 Problemi e criticità

Le query devono garantire:

- isolamento utente tramite filtro sull'email;
- atomicità delle operazioni complesse;
- aggiornamento coerente delle ridondanze;
- protezione dalla divisione per zero nelle statistiche;
- validità dei documenti rispetto alle sole spese dell'utente.

6.7.2 Inserimento atomico di una spesa

```
1 START TRANSACTION;
2
3 INSERT INTO PERIODO (Mese, Anno)
4 VALUES (?, ?)
5 ON DUPLICATE KEY UPDATE ID_Periodo = LAST_INSERT_ID(ID_Periodo);
6
7 INSERT INTO TRANSIZIONE
8 (TipoTransazione, Importo, Data, Descrizione, Email, ID_Categoria,
9  ID_Periodo, ID_Fonte)
10 VALUES ('S', ?, ?, ?, ?, ?, LAST_INSERT_ID(), NULL);
11
12 SET @id_spesa = LAST_INSERT_ID();
13
14 INSERT INTO SPESA_TAG (ID_Transizione, ID_Tag)
15 VALUES (@id_spesa, ?);
16
17 UPDATE BUDGET
18 SET Totale_Speso_Attuale = Totale_Speso_Attuale + ?
19 WHERE Email = ? AND ID_Periodo = ? AND ID_Categoria IS NULL;
20
21 UPDATE BUDGET
22 SET Totale_Speso_Attuale = Totale_Speso_Attuale + ?
```

```

22 WHERE Email = ? AND ID_Periodo = ? AND ID_Categoria = ?;
23
24 COMMIT;

```

Listing 1: Inserimento di una spesa con periodo, tag e budget

6.7.3 Upsert del budget

```

1 SELECT COALESCE(SUM(Importo), 0) AS Totale_Speso
2 FROM TRANSIZIONE
3 WHERE Email = ?
4     AND ID_Periodo = ?
5     AND TipoTransazione = 'S'
6     AND (? IS NULL OR ID_Categoria = ?);
7
8 INSERT INTO BUDGET
9 (Importo_Limite, Alert_Soglia, Totale_Speso_Attuale, ID_Periodo,
10  ID_Categoria, Email)
11 VALUES (?, ?, ?, ?, ?, ?)
12 ON DUPLICATE KEY UPDATE
13 Importo_Limite = VALUES(Importo_Limite),
14 Alert_Soglia = VALUES(Alert_Soglia),
15 Totale_Speso_Attuale = VALUES(Totale_Speso_Attuale);

```

Listing 2: Upsert del budget con totale iniziale coerente

6.7.4 Documenti dell'utente

```

1 SELECT D.ID_Documento, D.ID_Transizione, D.Path_File, D.Tipo_File,
2        D.Data_Acquisizione_Documento,
3        T.Data, T.Importo, T.Descrizione
4 FROM DOCUMENTO D
5 JOIN TRANSIZIONE T ON D.ID_Transizione = T.ID_Transizione
6 WHERE T.Email = ?
7     AND T.TipoTransazione = 'S'
8 ORDER BY T.Data DESC, D.ID_Documento DESC;

```

Listing 3: Documenti associati alle spese dell'utente

```

1 SELECT T.ID_Transizione, T.Data, T.Importo, T.Descrizione
2 FROM TRANSIZIONE T
3 LEFT JOIN DOCUMENTO D ON D.ID_Transizione = T.ID_Transizione
4 WHERE T.Email = ?
5     AND T.TipoTransazione = 'S'
6     AND D.ID_Documento IS NULL
7 ORDER BY T.Data DESC, T.ID_Transizione DESC;

```

Listing 4: Spese dell'utente ancora prive di documento

6.7.5 Analisi percentuale

```

1 SELECT
2     (
3         SUM(CASE WHEN TipoTransazione = 'S' THEN Importo ELSE 0 END)
4         /
5         NULLIF(SUM(CASE WHEN TipoTransazione = 'E' THEN Importo ELSE 0
6                     END), 0)

```

```

6      ) * 100 AS Percentuale_Spese_Su_Etrate
7 FROM TRANSIZIONE
8 WHERE Email = ? AND ID_Periodo = ?;

```

Listing 5: Rapporto spese/entrate mensile

6.7.6 Query di supporto

```

1 SELECT ID_Categoria, Nome, is_system
2 FROM CATEGORIA
3 WHERE is_system = TRUE OR Email_Proprietario = ?
4 ORDER BY is_system DESC, Nome;

```

Listing 6: Categorie disponibili per utente

```

1 SELECT T.ID_Transizione, T.Importo, T.Data, T.Descrizione,
2        COALESCE(C.Nome, F.Nome) AS Categoria_Fonte,
3        CASE WHEN T.TipoTransazione = 'S' THEN 'Spesa' ELSE 'Entrata' END
4        AS Tipo
5 FROM TRANSIZIONE T
6 LEFT JOIN CATEGORIA C ON T.ID_Categoria = C.ID_Categoria
7 LEFT JOIN FONTE F ON T.ID_Fonte = F.ID_Fonte
8 WHERE T.Email = ? AND T.Data BETWEEN ? AND ?
9 ORDER BY T.Data DESC;

```

Listing 7: Riepilogo transazioni per periodo

```

1 SELECT ID_Budget, Mese, Anno, Ambito,
2        Importo_Limite, Totale_Speso, Residuo, Superato
3 FROM v_budget_stato
4 WHERE Email = ?
5 ORDER BY Anno DESC, Mese DESC, Ambito;

```

Listing 8: Budget con residuo e stato di superamento

6.7.7 Motivazioni

L'uso di `PreparedStatement` evita concatenazioni insicure e permette di parametrizzare email, date e importi. L'uso di transazioni JDBC garantisce atomicità sulle operazioni che coinvolgono più tabelle. Le query sui documenti attraversano sempre `TRANSIZIONE`, così il filtro sull'utente rimane centralizzato.

7 Progettazione dell'applicazione

7.1 Architettura generale dell'applicazione

7.2 Tecnologie usate

7.3 Struttura dei package Java

7.4 Collegamento al database tramite JDBC

7.5 Organizzazione delle classi principali

7.6 Gestione degli errori e validazione input

7.7 Interfaccia grafica Java Swing

7.8 Schermate principali

7.9 Flussi d'uso principali

Registrazione studente

Inserimento di una spesa

Inserimento di una entrata

Definizione di un budget

Gestione di una spesa ricorrente

Associazione di un documento digitale

Visualizzazione statistiche amministratore

8 Test e validazione

8.1 Dati di test utilizzati

I dati di test includono due studenti normali e un amministratore. Lo studente `studente1@mail.com` ha transazioni, categorie personali, fonti, tag, budget e almeno un documento dimostrativo. Lo studente `studente2@mail.com` consente di verificare l'isolamento dei dati.

8.2 Test delle query SQL

Test	Input	Risultato atteso
Login valido	<code>studente1@mail.com</code> , <code>password1</code>	Utente trovato.
Login non valido	Password errata	Nessun utente trovato.
Documenti utente	Email studente 1	Solo documenti delle sue spese.
Spese senza documento	Email studente 1	Solo spese prive di documento.
Budget stato	Email studente 1	Budget ordinati per periodo e ambito.
Isolamento utenti	Email studente 2	Nessun dato personale dello studente 1.

8.3 Test delle funzionalità applicative

- avvio dell'applicazione da `Main`;
- login con credenziali demo;
- apertura della dashboard personale;
- visualizzazione della sezione documenti;

- apertura di un documento con path valido;
- gestione di path non valido con messaggio di errore;
- inserimento di un documento su una spesa senza documento.

8.4 Casi limite gestiti

- assenza di budget per il periodo corrente;
- assenza di documenti per un utente;
- file documento rimosso dal disco;
- assenza di entrate nel calcolo percentuale;
- tentativo di associare un documento a una spesa non appartenente all'utente.

8.5 Errori noti o limitazioni

La principale limitazione riguarda i documenti: il DB conserva il percorso del file, non il contenuto. Se il file viene spostato, cancellato o se l'utente usa un altro dispositivo, il percorso può risultare non valido. Questa scelta è consapevole e motivata dalla semplicità del progetto e dal minor carico sul database.

9 Conclusioni

9.1 Risultato ottenuto

Il progetto realizza una base di dati relazionale coerente con il dominio della gestione economica personale. L'applicazione Java consente di autenticare un utente, mantenere una sessione, interrogare dati filtrati per email e visualizzare le principali aree funzionali.

9.2 Scelte progettuali principali

Le scelte più rilevanti sono il collasso della gerarchia spesa/entrata in TRANSIZIONE, l'isolamento dati tramite email, la modellazione ibrida di elementi globali/personali, la ridondanza controllata del budget e la memorizzazione dei documenti tramite path.

9.3 Possibili sviluppi futuri

Sviluppi futuri includono:

- sincronizzazione o copia controllata dei documenti in una cartella applicativa;
- eventuale uso di `LONGLOB` per rendere i documenti portabili attraverso il solo database;
- completamento delle funzionalità amministrative;
- grafici statistici più avanzati;
- esportazione dei report in PDF o CSV.

A Script SQL completo

Gli script completi sono mantenuti nella cartella `doc/schemes`. In particolare, `schema.sql` contiene la versione pulita dello schema, mentre gli altri file raccolgono popolamento, trigger, viste e query operative.

B Query SQL complete

Le query principali sono riportate nel capitolo di implementazione SQL e nel file `Query_Operazioni_SAGE.sql`. Le query applicative usano parametri JDBC e filtrano sempre i dati personali tramite email.

C Screenshot dell'applicazione

Gli screenshot dell'applicazione possono essere inseriti in questa sezione durante la consegna finale. Le schermate più rilevanti sono login, dashboard, budget e gestione documenti.

D Manuale utente sintetico

Per usare l'applicazione occorre avviare `it.unibo.sage.Main`, effettuare il login con un utente presente nel database e navigare tra le sezioni della dashboard. I dati demo includono l'utente `studente1@mail.com` con password `password1`.

E Eventuale link al repository

Il link al repository può essere aggiunto in fase di consegna, insieme a eventuali istruzioni di compilazione ed esecuzione.